

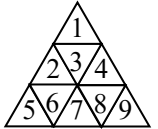
第六部分 树

6.1 排序二叉树

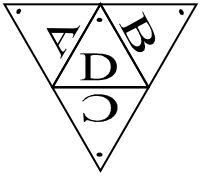
源程序名	tree.??? (pas, c, cpp)
可执行文件名	tree.exe
输入文件名	tree.in
输出文件名	tree.out

【问题描述】

一个边长为 n 的正三角形可以被划分成若干个小的边长为 1 的正三角形，称为单位三角形。如右图，边长为 3 的正三角形被分成三层共 9 个小的正三角形，我们把它们从顶到底，从左到右以 1~9 编号，见右图。同理，边长为 n 的正三角形可以划分成 n^2 个单位三角形。



四个这样的边长为 n 的正三角形可以组成一个三棱锥。我们将正三棱锥的三个侧面依顺时针次序（从顶向底视角）编号为 A, B, C，底面编号为 D。侧面的 A, B, C 号三角形以三棱锥的顶点为顶，底面的 D 号三角形以它与 A, B 三角形的交点为顶。左图为三棱锥展开后的平面图，每个面上标有圆点的是该面的顶，该图中侧面 A, B, C 分别向纸内方向折叠即可还原成三棱锥。我们把这 A, B, C, D 四个面各自划分成 n^2 个单位三角形。



对于任意两个单位三角形，如有一条边相邻，则称它们为相邻的单位三角形。显然，每个单位三角形有三个相邻的单位三角形。现在，把 1— $4n^2$ 分别随机填入四个面总共 $4n^2$ 个单位三角形中。

现在要求你编程求由单位三角形组成的最大排序二叉树。所谓最大排序二叉树，是指在所有由单位三角形组成的排序二叉树中节点最多的一棵树。对于任一单位三角形，可选它三个相邻的单位三角形中任意一个作为父节点，其余两个分别作为左孩子和右孩子。当然，做根节点的单位三角形不需要父节点，而左孩子和右孩子对于二叉树中的任意节点来说并不是都必须的。

【输入】

输入文件为 tree.in。其中第一行是一个整数 n ($1 \leq n \leq 18$)，随后的 $4n^2$ 个数，依次为三棱锥四个面上所填的数字。

【输出】

输出文件为 tree.out。其中仅包含一个整数，表示最大的排序二叉树所含的节点数目。

【样例】

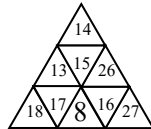
输入文件对应下图：



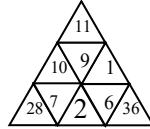
A



B



C



D

tree.in

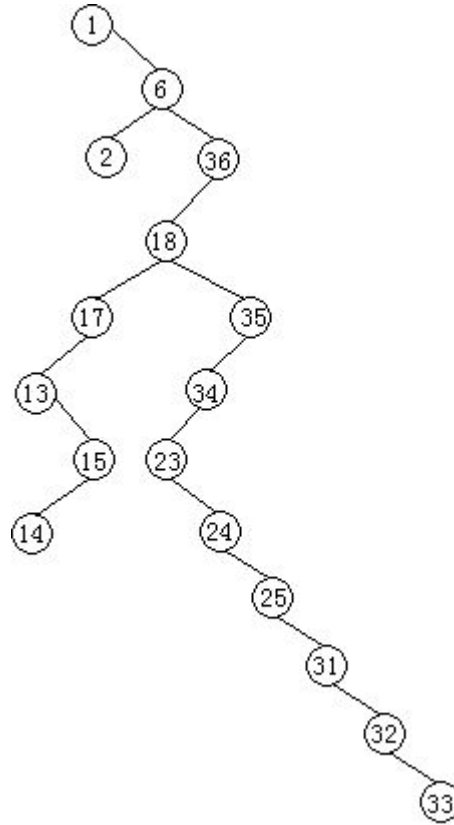
3 22 13 9
19 25 15 1
33 20 26 28
32 21 18 7
31 12 17 2

tree.out

17

29 24 8 6
3 23 16 36
5 34 27
4 35 11
30 14 10

输出样例文件对应的最大排序二叉树如下图所示：



【知识准备】

记忆化搜索的基本概念和实现方法。

【算法分析】

在讨论问题的解法之前，我们先来看看二叉排序树的性质。

二叉排序树是一棵满足下列性质的二叉树：

性质 1 它或是一棵空树，或是一棵二叉树，满足左子树的所有结点的值都小于根结点的值，右子树的所有结点的值都大于根结点的值；

性质 2 它若有子树，则它的子树也是二叉排序树。

根据性质 1，我们可以知道，二叉排序树的左右子树是互不交叉的。也就是说，如果确定了根结点，那么我们就可以将余下的结点分成两个集合，其中一个集合的元素可能在左子树上，另一集合的元素可能在右子树上，而没有一个结点同时可以属于两个集合。这一条性质，满足了无后效性的要求，正因为二叉排序树的左右子树是互不交叉的，所以如果确定根结点后，求得的左子树，对求右子树是毫无影响的。因此，如果要使排序树尽可能大，就必须满足左右子树各自都是最大的，即局部最优满足全局最优。

根据性质 2，二叉排序树的左右子树也是二叉排序树。而前面已经分析得到，左右子树也必须是极大的。所以，求子树的过程也是一个求极大二叉排序树的过程，是原问题的一个子问题。那么，求二叉排序树的过程就可以分成若干个阶段来执行，每个阶段就是求一棵极大的二叉排序子树。

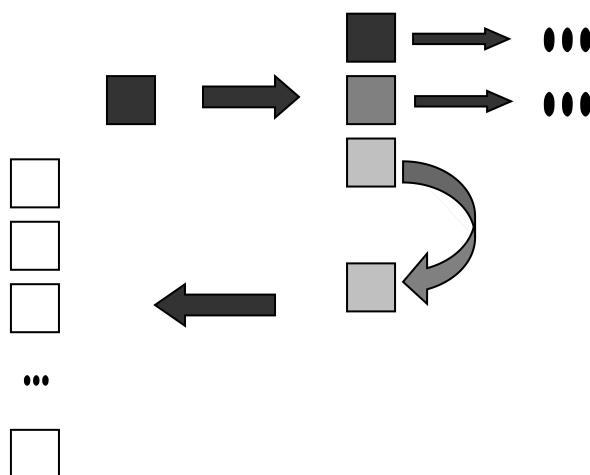
由此，我们看到，本题中，二叉排序树满足阶段性（性质 2）和无后效性（性质 1），可以用动态规划解决。

下面来看具体解决问题的方法。

不用说，首先必须对给出的正三棱锥建图，建成一张普通的无向图。

根据正三棱锥中结点的性质，每个结点均与三个结点相连。而根据二叉排序树的性质，当一个结点成为另一个结点的子结点后，它属于左子树还是右子树也就确定下来了。所以，可以对每个结点进行状态分离，分离出三种状态——该结点作为与它相连的三个结点的子结点时，所得的子树的状态。但是，一个子结点可以根据它的父结点知道的仅仅是该子树的一个界（左子树为上界，右子树为下界），还有一个界不确定，所以还需对分离出来的状态再进行状态分离，每个状态代表以一个值为界（上界或下界）时的子树状态。

整个分离过程如下图所示：



确定了状态后，我们要做的事就是推出状态转移方程。

前面已经提到，一个极大的二叉排序树，它的左右子树也必须是极大的。因此，如果我们确定以结点 n 为根结点，设所有可以作为它左子结点的集合为 N_1 ，所有可以作为它右子结点的集合为 N_2 ，则以 n 为根结点、结点大小在区间 $[l, r]$ 上的最大二叉排序树的结点个数为：

$$A(n, l, r) = \begin{cases} 0 & l > r \text{ 或 } n \notin [l, r] \\ 1 & l = r \\ \max_{n_1 \in N_1} \{A(n_1, l, n-1)\} + \max_{n_2 \in N_2} \{A(n_2, n+1, r)\} & n \in [l, r] \end{cases}$$

我们所要求的最大的二叉排序树的结点个数为：

$$MaxNodes = \max_{1 \leq i \leq n} \{A(i, 1, n)\}, \text{ 其中 } n \text{ 为结点的总个数}$$

从转移方程来看，我们定义的状态是三维的。那么，时间复杂度理应为 $O(n^3)$ 。其实并非如此。每个结点的状态虽然包含下界和上界，但是不论是左子结点还是右子结点，它的一个界取决于它的父结点，也就是一个界可用它所属的父结点来表示，真正需要分离的只有一维状态，要计算的也只是一维。因此，本题时间复杂度是 $O(n^2)$ （更准确的说应该是 $O(3n^2)$ ）。

此外，由于本题呈现一个无向图结构，如果用递推形式来实现动态规划，不免带来很大的麻烦。因为无向图的阶段性是很不明显的，尽管我们从树结构中分出了阶段。不过，实现动态规划的方式不仅仅是递推，还可以使用搜索形式——记忆化搜索。用记忆化搜索来实现

本题的动态规划可以大大降低编程复杂度。

6.2 树的重量

源程序名	weight.??? (pas, c, cpp)
可执行文件名	weight.exe
输入文件名	weight.in
输出文件名	weight.out

【问题描述】

树可以用来表示物种之间的进化关系。一棵“进化树”是一个带边权的树，其叶节点表示一个物种，两个叶节点之间的距离表示两个物种的差异。现在，一个重要的问题是，根据物种之间的距离，重构相应的“进化树”。

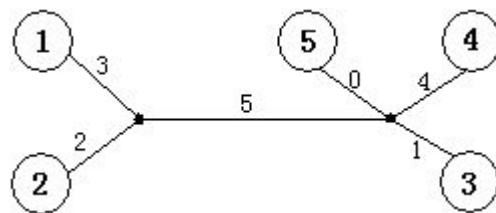
令 $N=\{1..n\}$ ，用一个 N 上的矩阵 M 来定义树 T 。其中，矩阵 M 满足：对于任意的 i, j, k ，有 $M[i,j]+M[j,k]\leq M[i,k]$ 。树 T 满足：

1. 叶节点属于集合 N ；
2. 边权均为非负整数；
3. $dT(i,j)=M[i,j]$ ，其中 $dT(i,j)$ 表示树上 i 到 j 的最短路径长度。

如下图，矩阵 M 描述了一棵树。

$$M = \begin{bmatrix} 0 & 5 & 9 & 12 & 8 \\ 5 & 0 & 8 & 11 & 7 \\ 9 & 8 & 0 & 5 & 1 \\ 12 & 11 & 5 & 0 & 4 \\ 8 & 7 & 1 & 4 & 0 \end{bmatrix}$$

树的重量是指树上所有边权之和。对于任意给出的合法矩阵 M ，它所能表示树的重量是惟一确定的，不可能找到两棵不同重量的树，它们都符合矩阵 M 。你的任务就是，根据给出的矩阵 M ，计算 M 所表示树的重量。下图是上面给出的矩阵 M 所能表示的一棵树，这棵树的总重量为 15。



【输入】

输入数据包含若干组数据。每组数据的第一行是一个整数 $n(2<n<30)$ 。其后 $n-1$ 行，给出的是矩阵 M 的一个上三角(不包含对角线)，矩阵中所有元素是不超过 100 的非负整数。输入数据保证合法。

输入数据以 $n=0$ 结尾。

【输出】

对于每组输入，输出一行，一个整数，表示树的重量。

【样例】

weight.in	weight.out
5	15
5 9 12 8	71

8 11 7
5 1
4
4
15 36 60
31 55
36
0

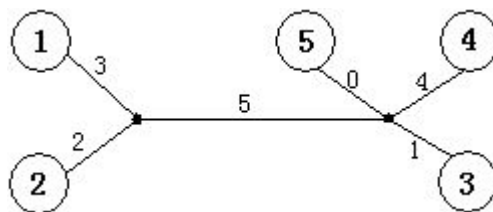
【知识准备】

树的基本特征。

【算法分析】

本题是一道涉及树性质的算法题，所以我们应该以树的性质为突破口，来讨论本题的算法。

首先来看一下简单的实例，就以题目中给出的例子来说明。下图所示的树，有 5 个叶子节点，两个内点，6 条边。我们已知的信息是任意两个叶子节点之间的距离，所以我们讨论的必然是叶子节点之间的关系，不可能涉及内点。

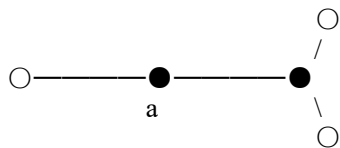


从图中我们可以看出，有些叶子节点是连在同一个内点上的，如①和②；也有些连在不同的内点上，如①和④。我们来看连在同一内点上的叶子节点有什么特殊的性质。就以①和②为例，①到③、④、⑤的距离分别为 9、12、8，②到③、④、⑤的距离分别为 8、11、7，正好都相差 1。这个“1”差在哪里呢？①连到内点的边长为 3，②连到内点的边长为 2，两者相差为 1。所以，相差的“1”正好就是两节点连到内点上的边长的差。再看①到②的距离，由于两叶子节点都是连在同一个内点上的，所以他们之间的距离，就是两者到内点的边长和。知道的边长和以及边长差，求出两边长就不难做到了。（注意：两叶子节点连到内点的边长是未知的）

再看一下不连在同一内点上的节点，①和④。①到②、③、⑤的距离分别为 5、9、8，④到②、③、⑤的距离分别为 11、5、4，没有一个统一的“差”。

其实，前面的结论都是非常直观而显然的，只不过关键是如何去利用。我们先来总结一下前面的结论：

- (1) 如果两叶子节点连在同一个内点上，则它们到其他叶子节点的距离有一个统一的“差”；
- (2) 如果两叶子节点连在不同的内点上，则它们到其他叶子节点的距离没有一个统一的“差”；



值得注意的是，图 6-3 中的 a 点不能算内点。我们所指的内点是至少连接两个叶子节点的点，像 a 这样的点完全可以去掉，不会影响树的权值和，如下图。

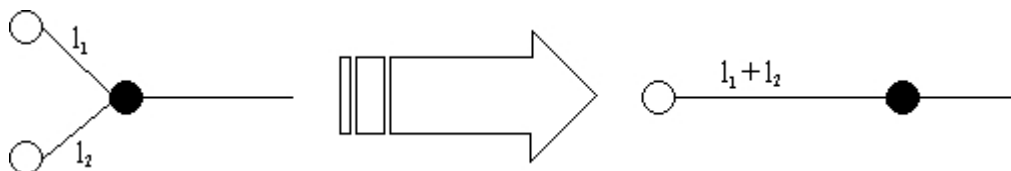




(3) 如果两叶子节点连在同一个内点上, 则它们之间的距离等于它们各自到内点的边长的和。

根据(1)和(2)两条性质, 很容易得到判断连接相同内点的两个叶子节点的方法, 即必须满足它们到其他所有叶子节点有统一的距离差(充分且必要)。

找到两个连接相同内点的叶子节点并计算出它们各自到内点的边长(不妨设为 l_1 和 l_2)以后, 我们可以作这样的操作: 删去一个节点, 令另一个节点到内点的边长为 $l_1 + l_2$ 。这样得到的新树, 权值和与原树相同, 但叶子节点少了一个, 如下图。



反复利用上述操作, 最后会得到一棵只有两个叶子节点的树, 这两个节点之间的边长就是原树的权值和。

算法需要反复执行 $n-2$ 次删除节点的操作。每次操作中, 需要枚举两个叶子节点, 并且还要有一个一维的判断, 时间复杂度是 $O(n^3)$ 的。所以, 整个算法的时间复杂度是 $O(n^4)$ 的。对于一个规模仅有 30 的题目来说, $O(n^4)$ 的算法足以解决问题了。当然, 算法本身应该还是有改进的余地的, 不过这里就不加以讨论了。

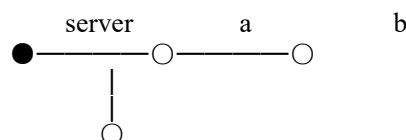
6.3 信号放大器

源程序名	booster.??? (pas, c, cpp)
可执行文件名	booster.exe
输入文件名	booster.in
输出文件名	booster.out

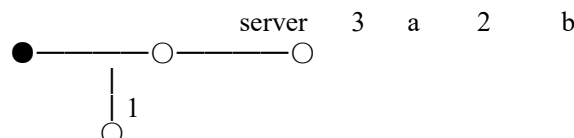
【问题描述】

树型网络是最节省材料的网络。所谓树型网络，是指一个无环的连通网络，网络中任意两个结点间有且仅有一条通信道路。

网络中有一个结点是服务器，负责将信号直接或间接地发送到各终端机。如图 6-4，server 结点发出一个信号给结点 a 和 c，a 再转发给 b。如此，整个网络都收到这个信号了。



但是，实际操作中，信号从一个结点发到另一个结点，会出现信号强度的衰减。衰减量一般由线路长度决定。



如上图，边上所标的数字为边的衰减量。假设从 server 出发一个强度为 4 个单位的信号，发到结点 a 后强度衰减为 $4-3=1$ 个单位。结点 a 再将其转发给结点 b。由于信号强度为 1，衰减量为 2，因此信号无法发送到 b。

一个解决这一问题的方法是，安装信号放大器。信号放大器的作用是将强度大于零的信号还原成初始强度（从服务器出发时的强度）。

上图中，若在结点 a 处安装一个信号放大器，则强度为 4 的信号发到 a 处，即被放大至 4。这样，信号就可以被发送的网络中的任意一个节点了。为了简化问题，我们假定每个结点只处理一次信号，当它第二次收到某个信号时，就忽略此信号。

你的任务是根据给出的树型网络，计算出最少需要安装的信号放大器数量。

【输入】

第一行一个整数 n ，表示网络中结点的数量。（ $n \leq 100000$ ）

第 $2 \sim n+1$ 行，每行描述一个节点的连接关系。其中第 $i+1$ 行，描述的是结点 i 的连接关系：首先一个整数 k ，表示与结点 i 相连的结点的数量。此后 $2k$ 个数，每两个描述一个与结点 i 相连的结点，分别表示结点的编号（编号在 $1 \sim n$ 之间）和该结点与结点 i 之间的边的信号衰减量。结点 1 表示服务器。

最后一行，一个整数，表示从服务器上出发信号的强度。

【输出】

一个整数，表示要使信号能够传遍整个网络，需要安装的最少的信号放大器数量。

如果不论如何安装信号放大器，都无法使信号传遍整个网络，则输出 “No solution.”

【样例】

booster.in	booster.out
4	1
2 2 3 3 1	
2 1 3 4 2	
1 1 1	
1 2 2	
4	

【问题分析】

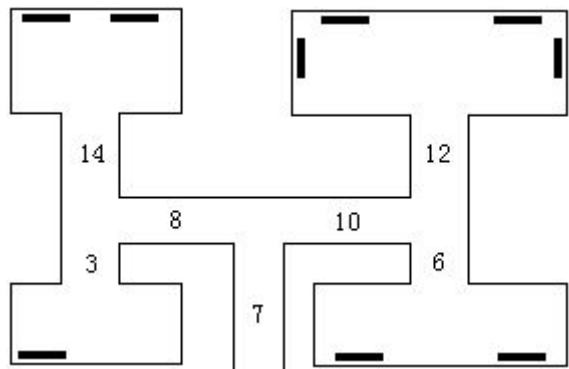
用贪心算法求解。从叶结点往根结点回推，当信号强度不够继续传送时，即增加一个信号放大器。算法的时间复杂度为 $O(n)$ 。

6.4 “访问”术馆

源程序名	gallery.??? (pas, c, cpp)
可执行文件名	gallery.exe
输入文件名	gallery.in
输出文件名	gallery.out

【问题描述】

经过数月的精心准备，Peer Brelstet，一个出了名的盗画者，准备开始他的下一个行动。艺术馆的结构，每条走廊要么分叉为两条走廊，要么通向一个展览室。Peer 知道每个展览室里藏画的数量，并且他精确测量了通过每条走廊的时间。由于经验老到，他拿下一幅画需要 5 秒的时间。你的任务是编一个程序，计算在警察赶来之前，他最多能偷到多少幅画。



【输入】

第 1 行是警察赶到的时间，以 s 为单位。第 2 行描述了艺术馆的结构，是一串非负整数，成对地出现：每一对的第一个数是走过一条走廊的时间，第 2 个数是它末端的藏画数量；如果第 2 个数是 0，那么说明这条走廊分叉为两条另外的走廊。数据按照深度优先的次序给出，请看样例。

一个展览室最多有 20 幅画。通过每个走廊的时间不超过 20s。艺术馆最多有 100 个展览室。警察赶到的时间在 10min 以内。

【输出】

输出偷到的画的数量。

【样例】

gallery.in (如图 6-6)	gallery.out
60	2
7 0 8 0 3 1 14 2 10 0 12 4 6 2	

【问题分析】

用树型动态规划求解。首先，题目保证数是二叉树。定义状态 $f(n, t)$ 表示在结点 n 所在子树上花 t 时间所能取到的最大分值，则状态转移方程为 $f(n, t) = \max \{f(\text{left}, t_0) + f(\text{right}, t - t_0)\}$ 其中 left 和 right 为 n 的左右子结点， t_0 的取值范围为 $[0, t]$ 。算法的时间复杂度为 $O(n^3)$ 。

6.5 聚会的快乐

源程序名	party.??? (pas, c, cpp)
可执行文件名	party.exe
输入文件名	party.in
输出文件名	party.out

【问题描述】

你要组织一个由你公司的人参加的聚会。你希望聚会非常愉快，尽可能多地找些有趣的热闹。但是劝你不要同时邀请某个人和他的上司，因为这可能带来争吵。给定 N 个人(姓名，他幽默的系数，以及他上司的名字)，编程找到能使幽默系数和最大的若干个人。

【输入】

第一行一个整数 N ($N < 100$)。接下来有 N 行，每一行描述一个人的信息，信息之间用空格隔开。姓名是长度不超过 20 的字符串，幽默系数是在 0 到 100 之间的整数。

【输出】

所邀请的人最大的幽默系数和。

【样例】

party.in	party.out
5	8
BART 1 HOMER	
HOMER 2 MONTGOMERY	
MONTGOMERY 1 NOBODY	
LISA 3 HOMER	
SMITHERS 4 MONTGOMERY	

【问题分析】

用动态规划求解。首先，很显然公司的人员关系构成了一棵树。设 $f[a]$ 为 a 参加会议的情况下，以他为根的子树取得的最大幽默值； $g[a]$ 为 a 不参加会议的情况下，以他为根的子树取得的最大幽默值。那么，状态转移方程就是：

$$f[a] = g[b_1] + g[b_2] + \dots + g[b_k] + 1$$

$$g[a] = \max(f[b_1], g[b_1]) + \max(f[b_2], g[b_2]) + \dots + \max(f[b_k], g[b_k])$$

其中 $b_1, b_2, \dots, b_k$ 为 a 的子结点。

最后的答案就是 $\max(f[\text{root}], g[\text{root}])$ ， root 是树根。

6.6 重建道路

源程序名	roads.??? (pas, c, cpp)
可执行文件名	roads.exe
输入文件名	roads.in
输出文件名	roads.out

【问题描述】

一场可怕的地震后，人们用 N 个牲口棚 ($1 \leq N \leq 150$ ，编号 $1..N$) 重建了农夫 John 的牧场。由于人们没有时间建设多余的道路，所以现在从一个牲口棚到另一个牲口棚的道路是惟一的。因此，牧场运输系统可以被构建成一棵树。John 想要知道另一次地震会造成多严重的破坏。有些道路一旦被毁坏，就会使一棵含有 P ($1 \leq P \leq N$) 个牲口棚的子树和剩余的牲口棚分离，John 想知道这些道路的最小数目。

【输入】

第 1 行：2 个整数， N 和 P

第 2.. N 行：每行 2 个整数 I 和 J ，表示节点 I 是节点 J 的父节点。

【输出】

单独一行，包含一旦被破坏将分离出恰含 P 个节点的子树的道路的最小数目。

【样例】

roads.in	roads.out
11 6	2
1 2	
1 3	
1 4	
1 5	
2 6	
2 7	
2 8	
4 9	
4 10	
4 11	

【样例解释】

如果道路 1-4 和 1-5 被破坏，含有节点 (1, 2, 3, 6, 7, 8) 的子树将被分离出来

【问题分析】

用树型动态规划求解。定义 $f(n, m)$ 为在 n 为根的子树中取 m 个节点的最小代价，则状态转移方程为：

$$f(n, m) = \min \{f(n_0, m_0) + f(n_1, m_1) + f(n_2, m_2) + \dots + f(n_k, m_k)\}$$

其中， $n_0, n_1, n_2, \dots, n_k$ 为 n 的 k 个儿子， $m_0 + m_1 + m_2 + \dots + m_k = m$ ，并且定义 $f(n_i, 0) = 1$ 。

最后的结果为：

$$\min \{f(\text{root}, p), \min \{f(n, p) \mid n \neq \text{root}\}\}$$

6.7 有线电视网

源程序名	tele.??? (pas, c, cpp)
可执行文件名	tele.exe
输入文件名	tele.in
输出文件名	tele.out

【问题描述】

某收费有线电视网计划转播一场重要的足球比赛。他们的转播网和用户终端构成一棵树状结构，这棵树的根结点位于足球比赛的现场，树叶为各个用户终端，其他中转站为该树的内部节点。

从转播站到转播站以及从转播站到所有用户终端的信号传输费用都是已知的，一场转播的总费用等于传输信号的费用总和。

现在每个用户都准备了一笔费用想观看这场精彩的足球比赛，有线电视网有权决定给哪些用户提供信号而不给哪些用户提供信号。

写一个程序找出一个方案使得有线电视网在不亏本的情况下使观看转播的用户尽可能多。

【输入】

输入文件的第一行包含两个用空格隔开的整数 N 和 M ，其中 $2 \leq N \leq 3000$ ， $1 \leq M \leq N-1$ ， N 为整个有线电视网的结点总数， M 为用户终端的数量。

第一个转播站即树的根结点编号为 1，其他的转播站编号为 2 到 $N-M$ ，用户终端编号为 $N-M+1$ 到 N 。

接下来的 $N-M$ 行每行表示一个转播站的数据，第 $i+1$ 行表示第 i 个转播站的数据，其格式如下：

$K \ A_1 \ C_1 \ A_2 \ C_2 \ \dots \ A_k \ C_k$

K 表示该转播站下接 K 个结点 (转播站或用户)，每个结点对应一对整数 A 与 C ， A 表示结点编号， C 表示从当前转播站传输信号到结点 A 的费用。最后一行依次表示所有用户为观看比赛而准备支付的钱数。

【输出】

输出文件仅一行，包含一个整数，表示上述问题所要求的最大用户数。

【样例】

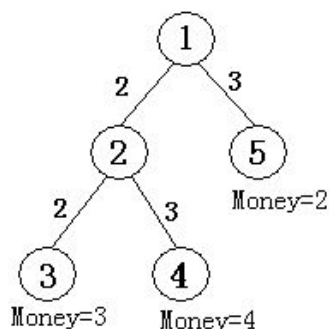
tele.in	tele.out
5 3	2
2 2 2 5 3	
2 3 2 4 3	
3 4 2	

【样例解释】

如右图所示，共有五个结点。结点①为根结点，即现场直播站，②为一个中转站，③④⑤为用户端，共 M 个，编号从 $N-M+1$ 到 N ，他们为观看比赛分别准备的钱数为 3、4、2，从结点①可以传送信号到结点②，费用为 2，也可以传送信号到结点⑤，费用为 3 (第二行数据所示)，从结点②可以传输信号到结点③，费用为 2。也可传输信号到结点④，费用为 3 (第三行数据所示)，如果要让所有用户 (③④⑤) 都能看上比赛，则信号传输的总费用为：

$2+3+2+3=10$ ，大于用户愿意支付的总费用 $3+4+2=9$ ，有线电视网就亏本了，而只让③④两个用户看比赛就不亏本了。

【问题分析】



用动态规划求解。状态这样定义，设 $F[A, M]$ 为 A 结点下支持 M 个用户所能得到的最大赢利。转移方程为：

$$F[A, M] = \text{Max} \{F[B_1, M_1] + F[B_2, M_2] + \cdots + F[B_k, M_k] \mid B_i \text{ 是 } A \text{ 的子结点, } M_1 + M_2 + \cdots + M_k = M\}$$

可以考虑将多叉数转换成二叉树来做，也可以考虑动态地分配内存。这样，空间复杂度约为 $O(n)$ ，时间复杂度为 $O(n^2)$ 。